DAT22-5

SQL & Data Querying

Querying, joining, and aggregating relational data

Translate a business question into SQL, execute it, and interpret the result.

Albert School — 28 hours

Contents

1 SELECT queries

A relational database stores data in tables of rows and columns. A `SELECT` query reads rows from a table. The clauses, in the order they are written:

```
SELECT columns
FROM table
WHERE condition
GROUP BY columns
HAVING group_condition
ORDER BY columns;
```

1.1 WHERE

`WHERE` filters individual rows by a condition. It runs before any grouping.

```
SELECT name, country, amount
FROM sales
WHERE country = 'France' AND amount > 100;
```

Conditions combine with `AND`, `OR`, `NOT`, and use `=`, `<>`, `<`, `>`, `<=`, `>=`, `BETWEEN`, `IN`, `LIKE`, and `IS NULL`.

1.2 ORDER BY

`ORDER BY` sorts the result. `ASC` is ascending (the default), `DESC` descending.

```
SELECT name, amount
FROM sales
ORDER BY amount DESC;
```

1.3 GROUP BY

`GROUP BY` collapses rows that share the same value in the listed columns into one group, so that an aggregate function can summarise each group.

```
SELECT country, SUM(amount)
FROM sales
GROUP BY country;
```

1.4 HAVING

`HAVING` filters groups by a condition on an aggregate, after grouping. `WHERE` filters rows before grouping; `HAVING` filters groups after.

```
SELECT country, SUM(amount)
FROM sales
GROUP BY country
HAVING SUM(amount) > 1000;
```

2 Aggregate functions

An aggregate function takes many rows and returns one value per group.

- `COUNT` — number of rows; `COUNT(*)` counts all rows, `COUNT(col)` counts non-`NULL` values, `COUNT(DISTINCT col)` counts distinct non-`NULL` values.
- `SUM` — total of a numeric column.
- `AVG` — mean of a numeric column.
- `MIN` — smallest value.
- `MAX` — largest value.

```
SELECT
  COUNT(*)      AS n_sales,
  SUM(amount)   AS total,
  AVG(amount)   AS mean,
  MIN(amount)   AS smallest,
  MAX(amount)   AS largest
FROM sales;
```

Each is appropriate for a different question: `COUNT` for how many, `SUM` for a total, `AVG` for a typical value, `MIN` and `MAX` for the extremes. `SUM` and `AVG` apply only to numeric columns; `COUNT`, `MIN`, and `MAX` apply to any type.

3 Joins

A join combines rows from two tables by matching values in a related column, usually a foreign key in one table referencing a primary key in the other.

```
SELECT o.id, c.name, o.amount
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.id;
```

The join type decides what happens to rows with no match:

- `INNER JOIN` — keeps only rows that match in both tables.
- `LEFT JOIN` — keeps all rows from the left table; unmatched right columns are `NULL`.
- `RIGHT JOIN` — keeps all rows from the right table; unmatched left columns are `NULL`.
- `FULL OUTER JOIN` — keeps all rows from both tables; unmatched columns on either side are `NULL`.

```
SELECT c.name, o.amount
FROM customers AS c
LEFT JOIN orders AS o ON o.customer_id = c.id;
```

A `LEFT JOIN` is the right choice when every row of the left table must appear even if it has no match — for example, listing all customers including those with no orders.

4 Subqueries and CTEs

A subquery is a `SELECT` nested inside another query. It can appear in `WHERE`, in `FROM`, or in the select list.

```
SELECT name, amount
FROM sales
WHERE amount > (SELECT AVG(amount) FROM sales);
```

A Common Table Expression (CTE), written with `WITH`, names a query so it can be referenced later. CTEs decompose a complex query into readable, named steps.

```
WITH country_totals AS (
  SELECT country, SUM(amount) AS total
  FROM sales
  GROUP BY country
)
SELECT country, total
FROM country_totals
WHERE total > 1000
ORDER BY total DESC;
```

Several CTEs are separated by commas and each can build on the previous ones.

5 Creating and manipulating tables

`CREATE TABLE` defines a table and the type of each column.

```
CREATE TABLE customers (
  id      INTEGER PRIMARY KEY,
  name    TEXT NOT NULL,
  country TEXT
);
```

`INSERT` adds rows, `UPDATE` changes existing rows, and `DELETE` removes rows.

```
INSERT INTO customers (id, name, country)
VALUES (1, 'Alice', 'France');

UPDATE customers
SET country = 'Germany'
WHERE id = 1;

DELETE FROM customers
WHERE id = 1;
```

An `UPDATE` or `DELETE` without a `WHERE` clause changes or removes every row in the table.

6 Database types

- A **relational database** stores data in related tables and is optimised for transactions — frequent reads and writes of individual rows. Appropriate for an application's operational data.
- A **data warehouse** stores large volumes of historical data optimised for analytical queries that aggregate across many rows. Appropriate for reporting and analytics.
- A **flat file store** keeps data in plain files such as CSV or JSON, with no query engine. Appropriate for small datasets, exchange between tools, or one-off storage.

7 Connecting from Python with sqlite3

SQLite is a relational database stored in a single file. Python's built-in `sqlite3` module connects to it and runs queries programmatically.

```
import sqlite3

conn = sqlite3.connect("shop.db")
cursor = conn.cursor()

cursor.execute(
    "SELECT name, amount FROM sales WHERE country = ?",
    ("France",),
)
rows = cursor.fetchall()
for row in rows:
    print(row)

conn.commit()   # save changes after INSERT/UPDATE/DELETE
conn.close()
```

`execute` runs a query; `fetchall` returns all result rows, `fetchone` returns one. Query parameters are passed with `?` placeholders rather than string formatting. After statements that change data, `commit` saves the changes.

8 Data quality at the query level

Common data quality issues can be found and fixed with queries.

- **Duplicates** — find them by grouping and counting, remove them with `DISTINCT` or by deleting the extra rows.

  ```
  SELECT email, COUNT(*)
  FROM customers
  GROUP BY email
  HAVING COUNT(*) > 1;
  ```

- **NULL handling** — `NULL` means unknown; it is tested with `IS NULL` / `IS NOT NULL` (not `=`), and replaced with `COALESCE(col, default)`.

  ```
  SELECT name, COALESCE(country, 'Unknown') AS country
  FROM customers;
  ```

- **Inconsistent string formats** — normalise with functions such as `TRIM` to remove surrounding spaces, `LOWER`/`UPPER` to standardise case.

  ```
  SELECT DISTINCT LOWER(TRIM(country)) AS country
  FROM customers;
  ```

9 From business question to SQL

Translating a business question into SQL means identifying the tables involved, the rows to filter (`WHERE`), the grouping (`GROUP BY`), the summary (aggregate functions), and the ordering, then interpreting the result in business terms.

For "Which three countries generated the most revenue last year?":

```
SELECT country, SUM(amount) AS revenue
FROM sales
WHERE year = 2025
```

```
GROUP BY country
ORDER BY revenue DESC
LIMIT 3;
```

The result — three country names with revenue totals — answers the question directly: these are the top three markets by revenue for the year.

10 Indexing

An index is an auxiliary data structure that lets the database find rows matching a column value without scanning the whole table.

```
CREATE INDEX idx_sales_country ON sales(country);
```

An index speeds up queries that filter or join on the indexed column, but slows down `INSERT`, `UPDATE`, and `DELETE` (the index must be maintained) and uses extra storage. Create an index on a column frequently used in `WHERE` or join conditions on a large table; do not index small tables or rarely filtered columns.

Exercises

1. Write a `SELECT` query using `WHERE`, `ORDER BY`, `GROUP BY`, and `HAVING` on a real database.
2. Use each of `COUNT`, `SUM`, `AVG`, `MIN`, and `MAX`, and state a question each one answers.
3. Join two tables with an `INNER JOIN`, then with a `LEFT JOIN`, and explain the difference in the results.
4. Rewrite a query that uses a subquery so that it uses a CTE instead.
5. Create a table, insert several rows, update one row, and delete another.
6. For a given scenario, state whether a relational database, a data warehouse, or a flat file store is appropriate, and why.
7. From Python, connect to a SQLite database with `sqlite3` and execute a parameterised query, printing the rows.
8. Write queries that find duplicate rows, replace `NULL` values, and normalise inconsistent string formatting.
9. Translate a business question into a SQL query, execute it, and write one sentence interpreting the result.
10. Create an index on a column and explain when it would and would not improve performance.